Loglog Counting of Large Cardinalities
(Extended Abstract)

Marianne Durand and Philippe Flajolet

Algorithms Project, INRIA–Rocquencourt, F78153 Le Chesnay (France)

Abstract. Using an auxiliary memory smaller than the size of this abstract, the LogLog algorithm makes it possible to estimate in a single pass and within a few percents the number of different words in the whole of Shakespeare's works. In general the LogLog algorithm makes use of m "small bytes" of auxiliary memory in order to estimate in a single pass the number of distinct elements (the "*cardinality*") in a file, and it does so with an accuracy that is of the order of $1/\sqrt{m}$. The "small bytes" to be used in order to count cardinalities till $N_{\max}$ comprise about $\log\log N_{\max}$ bits, so that cardinalities well in the range of billions can be determined using one or two kilobytes of memory only. The basic version of the LogLog algorithm is validated by a complete analysis. An optimized version, super–LogLog, is also engineered and tested on real-life data. The algorithm parallelizes optimally.

1 Introduction

The problem addressed in this note is that of determining the number of *distinct* elements, also called the *cardinality*, of a large file. This problem arises in several areas of data-mining, database query optimization, and the analysis of traffic in routers. In such contexts, the data may be either too large to fit at once in core memory or even too massive to be stored, being a huge continuous flow of data packets. For instance, Estan *et al.* [3] report traces of packet headers, produced at a rate of 0.5GB per hour of compressed data (!), which were collected while trying to trace a "worm" (Code Red, August 1 to 12, 2001), and on which it was necessary to count the number of *distinct* sources passing through the link. We propose here the LogLog algorithm that estimates cardinalities using only a very small amount of auxiliary memory, namely m memory units, where a memory unit, a "*small byte*", comprises close to $\log\log N_{\max}$ bits, with $N_{\max}$ an *a priori* upperbound on cardinalities. The estimate is (in the sense of mean values) asymptotically *unbiased*; the relative accuracy of the estimate (measured by a standard deviation) is close to $1.05/\sqrt{m}$ for our best version of the algorithm, Super–LogLog. For instance, estimating cardinalities till $N_{\max} = 2^{27}$ (a hundred million different records) can be achieved with $m = 2048$ memory units of 5 bits each, which corresponds to 1.28 kilobytes of auxiliary storage in total, the error observed being typically less than 2.5%. Since the algorithm operates incrementally and in a single pass it can be applied to data flows for which it provides on-line estimates available at any given time. Advantage can be taken

G. Di Battista and U. Zwick (Eds.): ESA 2003, LNCS 2832, pp. 605–617, 2003.
© Springer-Verlag Berlin Heidelberg 2003

of the low memory consumption in order to gather *simultaneously* a *very large number of statistics* on huge heterogeneous data sets. The LogLog algorithm can also be *fully distributed* or *parallelized*, with *optimum* speed-up and *minimal* interprocess communication. Finally, an embedded hardware design would involve strictly minimal resources.

Motivations. A traditional application of cardinality estimates is database query optimization. There, a complex query typically involves a variety of set-theoretic operations as well as projections, joints, and so on. In this context, knowing "for free" cardinalities of associated sets provides a valuable guide for selecting an efficient processing strategy best suited to the data at hand. Even a problem as simple as merging two large files with duplicates can be treated by various combinations of sorting, straight merging, and filtering out duplicates (in one or both of the files); the cost function of each possible strategy is then determined by the number of records as well as by the cardinality of each file. Probabilistic estimation algorithms also find a use in large data recording and warehousing environments. There, the goal is to provide an approximate response in time that is orders-of-magnitude less than what computing an exact answer would require: see the description of the Aqua Project by Gibbons *et al.* in [8].

The analysis of traffic in routers, as already mentioned, benefits greatly of cardinality estimators—this is lucidly exposed by Estan *et al.* in [2,3]. Certain types of attacks ("denial of service" and "port scans") are betrayed by alarmingly high counts of certain characteristic events in routers. In such situations, there is usually not enough resource available to store and search on-line the very large number of events that take place even in a relatively small time window.

Probabilistic counting algorithms can also be used within other algorithms whenever the final answer is the cardinality of a large set and a small tolerance on the quality of the answer is acceptable. Palmer *et al.* [13] describe the use of such algorithms in an extensive connectivity analysis of the internet topology. For instance, one of the tasks needed there is to determine, for each distance h, the number of pairs of nodes that are at distance at most h in the internet graph. Since the graph studied by [13] has close to 300,000 nodes, the number of pairs to be considered is well over 10^{10}, upon which costly list operations must be performed by exact algorithms. In contrast an algorithm that would be, in the abstract, suboptimal can be coupled with adapted probabilistic counting techniques and still provide reliable estimates. In this way, the authors of [13] were able to extract extensive metric information on the internet graph by keeping a reduced collection of data that reside in core memory. They report a reduction in run-time by a factor of more than 400.

Algorithms. The LogLog algorithm is probabilistic. Like in many similar algorithms, the first idea is to appeal to a hashing function in order to randomize data and bring them to a form that resembles random (uniform, independent) binary data. It is this hashed data set that is distilled into cardinality estimates by the algorithm. Various algorithms perform various tests on the hashed data set, then compare "*observables*" to what probabilistic analysis predicts, and finally "deduce" a plausible value of the parameter of interest. In the case of